COMP9414

Assignment 1: Forecasting Solar Radiation with Neural Networks

Term 1, 2026

Due: Friday, 20 March 2026, 5:00 PM AEST
Worth: 20% of final grade
Submission: Electronic submission via Moodle

Introduction

This assignment explores neural network techniques for solar radiation forecasting, a critical problem in renewable energy management and climate science. Solar radiation prediction is essential for grid stability, energy storage optimisation, and resource planning in solar power systems worldwide.

You will build and evaluate neural network architectures to predict daily solar radiation (W/m^2) using historical data with quality control flags. Starting with comprehensive data preprocessing including Fourier cyclical encoding and rolling statistics, you will progress to multi-output models with custom loss functions, transfer learning strategies, and techniques including uncertainty quantification and attention mechanisms.

Background

Solar Radiation and Energy Systems

Solar radiation is the electromagnetic energy emitted by the sun that reaches Earth's surface. Measured in watts per square metre (W/m^2), it is the fundamental driver of solar power generation and has profound impacts on weather patterns, climate systems, and agricultural productivity. Accurate forecasting of solar radiation is crucial for grid integration and stability of variable renewable energy supply, optimising battery charging and discharging schedules, forecasting energy production and market prices, and understanding long-term climate trends and variability.

Solar radiation exhibits complex temporal patterns driven by seasonal variations, short-term autocorrelation, and stochastic cloud cover effects. Traditional physics-based models require extensive atmospheric data and computational resources. Neural networks offer a data-driven alternative capable of learning these complex patterns from historical observations.

Data Quality and Preprocessing

Real-world solar radiation data often contains quality issues including sensor malfunctions, calibration errors, and missing values. The dataset includes a `bad_quality` flag (0 = good, 1 = bad) that must be handled appropriately during preprocessing. Additionally,

the cyclical nature of daily and seasonal patterns requires specialised encoding techniques such as Fourier transformations to help models understand temporal relationships.

The Neural Network Approach

This assignment emphasises neural network techniques beyond basic architectures. You will implement custom loss functions, multi-output models that leverage shared representations, transfer learning strategies, uncertainty quantification methods, and attention mechanisms for interpretability. For a comprehensive overview of deep learning approaches to time-series forecasting, see [1].

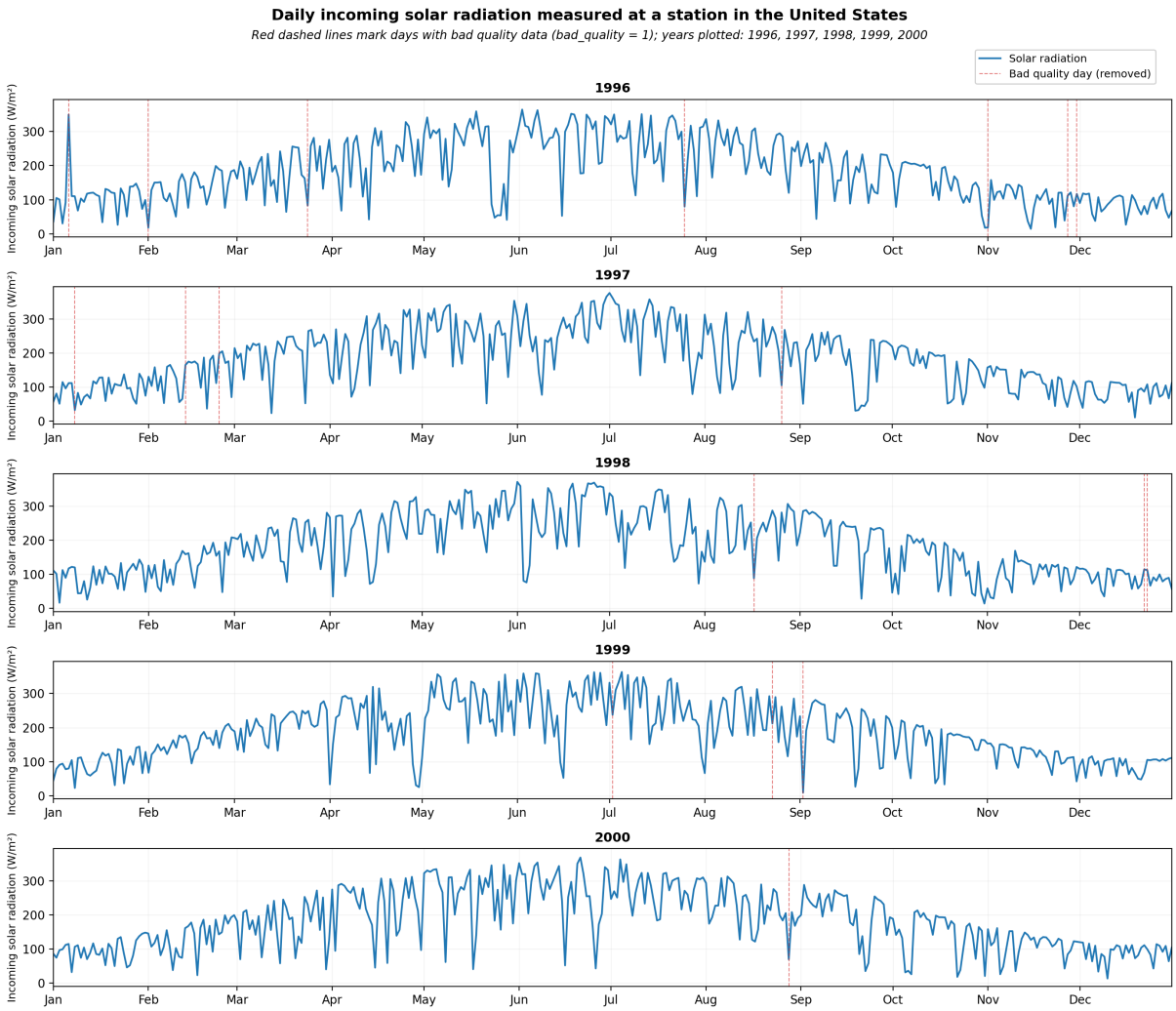

Figure 1: Example: Daily solar radiation time-series showing seasonal patterns.

Assignment Specification

Data Description

Daily solar radiation data is provided in files named `daily_solar_YYYY.csv` for years 1996–2013 (see Table 1).

Table 1: Structure of `daily_solar_YYYY.csv` Dataset

Column Name	Description	Unit
date	Calendar date (YYYY-MM-DD)	–
solar_t	Daily solar radiation at time t	W/m^2
bad_quality	Quality flag (0=good, 1=bad)	–

Data Split: Training covers 1996–2010, spanning 15 years. Validation is a subset of the training data, where the split ratio is your choice. Test covers 2011–2013, spanning 3 years. Hidden data covers 2014–2019 and will be provided prior to the discussion session. Use a **temporal split**, meaning you should use the most recent portion of the training years for validation rather than random sampling, to avoid temporal leakage. At each split boundary, drop the first 7 rows of the subsequent partition so that no lagged or rolling feature uses observations from the previous partition.

Assignment Tasks Overview

This assignment consists of five parts with increasing complexity. Each part is assessed on both implementation and understanding demonstrated during discussion; see Table 2 for details.

Part A: Data preprocessing with quality control, Fourier encoding, and lagged features.

Part B: Multi-output model with loss function comparison.

Part C: Individual models and transfer learning strategies.

Part D: Uncertainty quantification, attention, and ablation.

Part E: Statistical analysis and discussion.

Part A: Data Preprocessing

Quality Control and Feature Engineering

1. **Load and Clean Data**: Load all daily solar radiation files (1996–2013). Remove rows where `bad_quality` = 1. Report the percentage of rows removed relative to the total dataset.

2. **Fourier Cyclical Encoding**: The solar radiation exhibits strong daily and seasonal cycles. Encode the day-of-year using Fourier features:

   - $\text{day_sin} = \sin\left(\frac{2\pi \times \text{day_of_year}}{365}\right)$
   - $\text{day_cos} = \cos\left(\frac{2\pi \times \text{day_of_year}}{365}\right)$

   These features help the model understand that day 365 is adjacent to day 1. For leap years, use 366 as the denominator.

3. **Rolling Statistics**: Create **backward-looking** rolling window features over the preceding 7 valid observations (using only past data available at time t). After

quality filtering, some calendar days are missing; the rolling window operates on consecutive rows in the filtered dataset, not calendar days:

- Rolling mean: $\text{solar_mean_7d} = \frac{1}{7}\sum_{i=0}^{6} \text{solar}_{t-i}$
- Rolling standard deviation (sample, ddof = 1):

$$\text{solar_std_7d} = \sqrt{\frac{1}{6}\sum_{i=0}^{6} (\text{solar}_{t-i} - \text{solar_mean_7d})^2}$$

- Rolling minimum: $\text{solar_min_7d} = \min_{i\in\{0,\ldots,6\}} \text{solar}_{t-i}$
- Rolling maximum: $\text{solar_max_7d} = \max_{i\in\{0,\ldots,6\}} \text{solar}_{t-i}$

Rows with fewer than 7 preceding observations should be dropped. Do **not** use forward-looking windows as this would constitute data leakage.

4. **Lagged Features**: Create lagged features for $t-1$ through $t-7$. As with rolling statistics, lags refer to preceding rows in the filtered dataset:

$$\text{solar}_{t-k} \text{ for } k \in \{1, 2, 3, 4, 5, 6, 7\}$$

5. **Target Variables**: Create target variables for $t+1$ through $t+7$:

$$\text{solar}_{t+k} \text{ for } k \in \{1, 2, 3, 4, 5, 6, 7\}$$

6. **Scaling**: Apply z-score standardisation to all *features* (not targets). Targets remain in original units (W/m^2). **Important**: Fit the scaler on the *training set only*, then transform validation and test sets using the same fitted scaler. Save scaler objects (e.g. via `joblib` or `pickle`) for use during hidden-set evaluation.
7. **Visualisation**: Produce the following plots:
   - A time-series plot of daily solar radiation over the full date range, with bad-quality days visually marked.
   - A histogram of solar radiation values after quality filtering.
   - A summary of the quality flag distribution (e.g. a bar chart of good vs bad counts per year).

Part B: Multi-Output Model with Custom Loss Function

In this part you will build a single multi-output neural network and study how the choice of loss function affects forecasting performance. You will train **3 models total**, all using the same architecture and training protocol, changing only the loss function. This controlled comparison isolates the effect of the loss.

Architecture

8. **Multi-Output Network**: Build a feedforward neural network. The following reference architecture is provided as a starting point:

   - Input layer: number of features from Part A
   - Hidden layers: 2 layers with 64 neurons each
   - Activation: ReLU for hidden layers
   - Output layer: 7 neurons with linear activation (do not change)

   You may modify the hidden layer depth, width, or activation functions (e.g., 1–4 hidden layers, 32–256 neurons per layer) provided you briefly justify your choices. The reference architecture above is sufficient for full marks; no architecture search is required. **Important**: your chosen architecture must remain a feedforward MLP with the same 7-neuron linear output layer, must be used identically for all three loss function experiments below, and will be reused throughout Parts C and D. You will train this architecture 15+ times across the assignment, so choose one that trains quickly.

Loss Function Comparison

Train your chosen architecture three times, once with each of the following loss functions. Use the same random seed, learning rate, batch size, and early stopping configuration for all three so the comparison is fair.

8.1. Weighted MSE (baseline):

$$L_{\text{MSE}} = \sum_{k=1}^{7} w_k \cdot \text{MSE}_k$$

where $\text{MSE}_k = \frac{1}{n}\sum_{i=1}^{n}(y_{i,k} - \hat{y}_{i,k})^2$ and $w_k = 1/\sqrt{k}$.

8.2. Pseudo-Huber Loss (smooth robust alternative):

$$L_{\text{PH}} = \sum_{k=1}^{7} w_k \cdot \frac{1}{n}\sum_{i=1}^{n} \delta^2 \left(\sqrt{1 + \left(\frac{u_{i,k}}{\delta}\right)^2} - 1\right) \quad \text{with } \delta = 50.0$$

where $u_{i,k} = y_{i,k} - \hat{y}_{i,k}$.

8.3. Weighted Quantile Loss (asymmetric pinball loss):

$$L_{\text{QL}} = \sum_{k=1}^{7} w_k \cdot \frac{1}{n}\sum_{i=1}^{n} \rho_\tau(y_{i,k} - \hat{y}_{i,k}) \quad \text{with } \tau = 0.8$$

where $\rho_\tau(u) = u(\tau - \mathbb{1}_{u<0})$ is the quantile check function and $\mathbb{1}_{u<0}$ is the indicator function that equals 1 when $u < 0$ and 0 otherwise.

Training and Evaluation

9. **Training**: For each of the 3 models, plot training and validation loss curves. Use early stopping with a patience of at least 15 epochs.

10. **Evaluation**: Evaluate all 3 models on the test set (2011–2013) using the following metrics **per forecast horizon**: RMSE, MAE, MBE, and P95 (see Evaluation Metrics section for definitions). Include a macro-average row across all 7 horizons. Present the results in a comparison table and a grouped bar chart comparing all four metrics (RMSE, MAE, MBE, P95) by horizon across the three loss functions. Note that MBE reveals directional bias (especially visible with the asymmetric Quantile loss) and P95 reveals tail-risk differences (where Pseudo-Huber's robustness may appear).

Part C: Individual Models and Transfer Learning

Individual Model Development

11. **Seven Separate Models**: As a baseline, build 7 independent single-output neural networks, each predicting a single forecast horizon ($t + 1$ to $t + 7$). Use standard MSE loss and the same architecture and training hyperparameters for all 7 models; vary only the target horizon.

12. **Transfer Learning Strategies**: Implement and compare three transfer learning approaches (for background on transfer learning, see [2]):

    12.1. **Strategy 1 - Sequential Fine-tuning**: Train a model for $t + 1$, then use its weights as initialisation for $t + 2$, and so on.

    12.2. **Strategy 2 - Feature Extraction**: Train a single-output model for $t + 1$. For transfer to $t + 7$, freeze all parameters except the final output layer and fine-tune only that layer on the $t + 7$ target. Report results for $t + 7$ only.

    12.3. **Strategy 3 - Multi-task Learning**: Train a shared encoder with *separate* task-specific output heads (each with at least one hidden layer) for each horizon. Note: this differs from the Part B multi-output model, which uses a single shared network with a 7-neuron output layer. Here, each horizon has its own output sub-network branching from the shared encoder.

13. **Comparison**: Create a table comparing the Part B multi-output baseline, the seven separate models (Item 11), Strategy 1, and Strategy 3 in terms of RMSE, MAE, and wall-clock training time per forecast horizon. Include a grouped bar chart comparing RMSE (and optionally MAE) by horizon across all models. For Strategy 2, report the $t + 7$ metrics separately (since it is only applied to that horizon).

Part D: Advanced Techniques

Uncertainty, Attention, and Ablation

14. **Uncertainty Quantification via Monte Carlo Dropout**: Build a multi-output model using your Part B architecture but with dropout layers (rate 0.1–0.3) after

each hidden layer [3]. Do not use batch normalisation in this model, as it interacts poorly with MC Dropout at inference. At inference, keep dropout active and run $T \geq 30$ stochastic forward passes to estimate prediction uncertainty:

$$\hat{\mu}_i = \frac{1}{T}\sum_{t=1}^{T} f_t(x_i), \quad \hat{\sigma}_i^2 = \frac{1}{T}\sum_{t=1}^{T}(f_t(x_i) - \hat{\mu}_i)^2$$

where T is the number of forward passes and f_t denotes the t-th stochastic evaluation. Report RMSE and MAE per forecast horizon (with macro-average) for the MC Dropout predictive mean. Report calibration using a binned plot: group predictions by predicted standard deviation and, for each bin, plot the mean predicted standard deviation against the realised RMSE.

15. **Attention Mechanism with Visualisation**: Implement a temporal attention mechanism that learns to weight different time steps [4]. Reshape the 7 lagged features into a sequence of length 7, each with embedding dimension d_k, then apply scaled dot-product attention. Concatenate the attention output with the remaining (non-lag) features before passing through the output layers:

$$\text{attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where Q, K, V are linear projections of the lagged input sequence. The attention is shared across all forecast horizons. Report RMSE and MAE per forecast horizon (with macro-average). Visualise the 7×7 attention weight matrix as a heatmap for at least 5 sample predictions to show which lags attend to which other lags.

16. **Ablation Study**: Using the attention model (Item 15) as the baseline, systematically remove components and measure performance impact:

    - Remove Fourier features
    - Remove rolling statistics
    - Remove lagged features
    - Remove attention mechanism

    Create a bar chart showing RMSE and MAE degradation for each ablation. **Hint**: When sequence-based features are removed (e.g. lagged features), the attention mechanism cannot operate; fall back to a standard feedforward network for that ablation.

Part E: Statistical Analysis and Discussion

Comprehensive Analysis

17. **Statistical Significance Testing**: Perform paired t-tests on per-day mean absolute errors (averaged across all 7 horizons for each day in the test set). The models to compare are defined as follows:

    - **Multi-output**: Part B model (Item 8, trained with Item 8.1).

- **Individual**: Part C seven separate models (Item 11)
- **Attention**: Part D attention model (Item 15)
- **Multi-task**: Part C Strategy 3 (Item 12.3)

Compare at least the following pairs: (i) multi-output vs individual, (ii) attention vs multi-output, (iii) multi-output vs multi-task. Use:

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}$$

where $\bar{d}$ is the mean paired difference, s_d is its standard deviation, and n is the number of test samples. Report t-statistics, p-values, and 95% confidence intervals for each comparison.

18. **Error Analysis**: For each of the following dimensions, report grouped RMSE and MAE (computed on the test set by averaging over the 7 forecast horizons per day, then grouping days by the dimension):

    - Seasonal patterns: assign season by calendar month. The data is from a Northern Hemisphere location, so use: winter = Dec–Feb, spring = Mar–May, summer = Jun–Aug, autumn = Sep–Nov
    - Forecast horizon: group by short-term ($t+1$ to $t+3$) vs long-term ($t+4$ to $t+7$)
    - Weather conditions: classify days as "clear" (solar radiation > 75th percentile of training data) vs "cloudy" (< 25th percentile) vs "mixed" (between)
    - Magnitude of solar radiation: split into terciles (low / medium / high) based on training data distribution

    Produce a grouped bar chart for each dimension showing both RMSE and MAE.

Evaluation Metrics

The following metrics are used throughout the assignment. RMSE and MAE are core metrics required in every part. MBE and P95 are additional diagnostics required only in Part B, where they help reveal how different loss functions affect bias and tail behaviour.

Core Metrics

- **Root Mean Square Error (RMSE)**:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

- **Mean Absolute Error (MAE)**:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Part B Additional Metrics

- **Mean Bias Error (MBE)**:

$$\text{MBE} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)$$

- **95th-Percentile Absolute Error (P95)**:

$$\text{P95} = \text{Percentile}_{95}(\{|y_i - \hat{y}_i|\}_{i=1}^{n})$$

Assessment and Evaluation

Evaluation Process

Your assignment will be evaluated during a scheduled discussion session with your tutor. Attendance is compulsory. Submissions not discussed will receive zero marks.

Prior to the discussion session, you will be provided with hidden test data covering years 2014–2019, in the same per-year CSV format as the training data (i.e. `daily_solar_2014.csv` through `daily_solar_2019.csv`). You must create an evaluation script that:

1. Loads the hidden dataset
2. Applies preprocessing (using saved scalers)
3. Loads all trained models
4. Generates evaluation metrics (RMSE and MAE per forecast horizon, with macro-averages)
5. Produces visualisations (predictions vs observations, error distributions)

During the discussion, you will run this evaluation script on the hidden test dataset, demonstrating that your models work correctly on unseen data.

Assessment Breakdown

Table 2: Assignment Assessment Breakdown

Assessment Component	Marks
Assignment (20 marks)	
Implementation Components (8 marks)	
Part A: Data preprocessing with quality control and Fourier encoding	1.5
Part B: Multi-output model with custom loss function	2
Part C: Individual models and transfer learning strategies	1.5
Part D: Advanced techniques (uncertainty, attention, ablation)	2
Part E: Statistical analysis and discussion	1
Understanding & Discussion (12 marks)	
Your tutor will select one topic from each group (3 topics total). Each is worth 4 marks.	
Group 1 – Foundations	4
1. Data preprocessing and feature engineering rationale	
2. Custom loss function implementation and comparison	
Group 2 – Architecture	4
3. Multi-output vs individual model comparison	
4. Transfer learning strategy effectiveness	
Group 3 – Advanced	4
5. Advanced techniques (uncertainty, attention, ablation)	
6. Statistical analysis and error interpretation	
Total	**20**

Submission Requirements

Submit the following files in a single ZIP archive named `zID_assignment1.zip` via **Moodle**:

1. **Jupyter Notebook** (`zID_assignment1.ipynb`): Must show ALL output cells executed with all plots and results visible (except the hidden dataset evaluation cell, which should be included but may be unexecuted at submission time). Include:
   - Part A: Data preprocessing, visualisations, and quality control statistics
   - Part B: Multi-output model architecture, loss function comparison, training curves
   - Part C: Individual models and transfer learning comparison
   - Part D: Uncertainty quantification, attention visualisations, ablation study
   - Part E: Statistical tests, error analysis, discussion
2. **Trained Models and Scalers**: Include all trained model files and preprocessing scalers. At a minimum:
   - 1 multi-output model (Part B)

- 7 individual models (Part C, Item 11)
- Transfer learning models: Strategy 1 final model, Strategy 2 fine-tuned model, Strategy 3 multi-task model
- MC Dropout model and Attention model (Part D)
- All preprocessing scalers

3. **Evaluation Cell**: Include a dedicated evaluation cell that loads the hidden test dataset, applies preprocessing, loads all models, and generates evaluation metrics and visualisations. Before submission, verify this cell works correctly by running it on your test set (2011–2013). **If this cell is not included in your Jupyter notebook, 2 marks will be deducted from your total assignment mark.**

Important: Your submission should contain **at least 15 files**, all of which must load and run correctly during evaluation. Failure to do so may result in losing up to 50% of the marks for that component.

Deadline and Questions

Deadline: Week 5, Friday 20 March 2026, 5:00 PM AEST

Late submissions: A penalty of 5% of the maximum mark per day applies to late submissions (e.g., 1 day late = 5% penalty, 2 days = 10%, etc.). Submissions more than 5 days late will receive a mark of zero. If you have an approved special consideration, your deadline will be adjusted accordingly and no late penalty will apply.

Please use the forum on Moodle to ask questions related to the assignment. We will prioritise questions asked in the forum. However, you should not share your code to avoid plagiarism. If necessary, email f.cruz@unsw.edu.au or m.mesto@unsw.edu.au.

Although we try to answer questions as quickly as possible, we might take up to 1–2 business days to reply. Last-moment questions might not be answered timely.

For questions regarding discussion sessions, please contact your tutor directly (see Table 3).

Table 3: Tutorial Sessions and Tutors

Code	Section	Time	Tutor	Email
4263	H09A	Thu 09–11 (Wk 1–10)	Adam Stucci	a.stucci@unsw.edu.au
4265	H11A	Thu 11–13 (Wk 1–10)	John Chen	xin.chen9@student.unsw.edu.au
4267	H13A	Thu 13–15 (Wk 1–10)	Stephen Xie	xiongyu.xie@student.unsw.edu.au
4268	H13B	Thu 13–15 (Wk 1–10)	Oltan Sevinc	m.sevinc@unsw.edu.au
4269	H15A	Thu 15–17 (Wk 1–10)	Chunyang Meng	chunyang.meng@unsw.edu.au
4270	H15B	Thu 15–17 (Wk 1–10)	Joffrey Ji	joffrey.ji@unsw.edu.au
4271	H18A	Thu 18–20 (Wk 1–10)	Ishan Dubey	i.dubey@student.unsw.edu.au
4272	H18B	Thu 18–20 (Wk 1–10)	John Chen	xin.chen9@student.unsw.edu.au
4273	H18C	Thu 18–20 (Wk 1–10)	Joffrey Ji	joffrey.ji@unsw.edu.au
4274	W18A	Wed 18–20 (Wk 1–10)	Hadha Afrisal	hadha.afrisal@unsw.edu.au
4275	W18B	Wed 18–20 (Wk 1–10)	Haitao Gao	haitao.gao@student.unsw.edu.au
4253	F09A	Fri 09–11 (Wk 1–6,8–10)	Khaja Ahmed Shaik	k.shaik@student.unsw.edu.au
4255	F11A	Fri 11–13 (Wk 1–6,8–10)	Hadha Afrisal	hadha.afrisal@unsw.edu.au
4256	F11B	Fri 11–13 (Wk 1–6,8–10)	Peter Ho	peter.ho2@student.unsw.edu.au
4257	F14A	Fri 14–16 (Wk 1–6,8–10)	Peter Ho	peter.ho2@student.unsw.edu.au
4262	F16C	Fri 16–18 (Wk 1–6,8–10)	Stephen Xie	xiongyu.xie@student.unsw.edu.au

Academic Integrity

This assignment must be completed individually. Do not share your code or solutions with other students, and do not post assignment content on external forums.

Permitted: Discussion of concepts and approaches with other students; consultation with course staff; use of course materials and textbooks; AI tools for learning concepts and debugging (not complete solutions). You are responsible for understanding and being able to explain every line of your submitted code.

Not Permitted: Sharing or copying code between students; submitting code written by others; posting assignment content online.

References

[1] Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: a survey. *Philosophical Transactions of the Royal Society A*, 379(2194):20200209, 2021.

[2] Fuzhen Zhuang, Zhiyuan Qi, Keyu Duan, Dongbo Xi, Yongchun Zhu, Hengshu Zhu, Hui Xiong, and Qing He. A comprehensive survey on transfer learning. *Proceedings of the IEEE*, 109(1):43–76, 2021.

[3] Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 1050–1059, 2016.

[4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.